Course Title : HIGH PERFORMANCE COMPUTING ASS-9

Name: V.Ranjith
Hall Ticket No:2303A51734
Batch-11

Lab: OpenMP Synchronization and Load Imbalance Analysis

Objective:

To analyze synchronization overhead and load imbalance in OpenMP programs and optimize performance using proper directives.

Task 1: Synchronization Overhead (Critical vs Reduction)

Step 1:

Create a file named reduction_bad.c

Implement parallel summation using:

```
#pragma omp critical
```

inside the loop.

Compile:

```
gcc -O2 -fopenmp reduction_bad.c -o reduction_bad
```

Run with:

```
export OMP_NUM_THREADS=1
export OMP_NUM_THREADS=4
```

Record execution time.

Step 2:

Create reduction_good.c

Replace critical with:

```
#pragma omp parallel for reduction(+:sum)
```

Compile and run with same thread counts.

```
nithish@LAPTOP-A2FOJ865: ~
  GNU nano 7.2                                   reduction_bad.c *
#include <stdio.h>
#include <omp.h>

int main() {
    long long i;
    long long N = 100000000;
    double sum = 0.0;

    double start = omp_get_wtime();

    #pragma omp parallel for
    for(i = 0; i < N; i++) {

        double value = i * 0.5;

        // synchronization using critical
        #pragma omp critical
        {
            sum += value;
        }
    }

    double end = omp_get_wtime();

    printf("Sum = %f\n", sum);
    printf("Execution Time (critical) = %f seconds\n", end - start);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was
```

```
nithish@LAPTOP-A2FOJ865:~$ nano reduction_bad.c
nithish@LAPTOP-A2FOJ865:~$ ls
numaass  reduction_bad.c  sum_openmp  sum_openmp.c  sum_serial  sum_serial.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp reduction_bad.c -o reduction_bad
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=1
./reduction_bad
Sum = 2499999975000000.000000
Execution Time (critical) = 1.322864 seconds
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./reduction_bad
Sum = 2499999975000000.000000
Execution Time (critical) = 10.316146 seconds
nithish@LAPTOP-A2FOJ865:~$
```

Step 2:

```
nithish@LAPTOP-A2FOJ865: ~
  GNU nano 7.2                                   reduction_good.c *
#include <stdio.h>
#include <omp.h>

int main() {
    long long i;
    long long N = 100000000;
    double sum = 0.0;

    double start = omp_get_wtime();

    // reduction removes critical overhead
    #pragma omp parallel for reduction(+:sum)
    for(i = 0; i < N; i++) {
        sum += i * 0.5;
    }

    double end = omp_get_wtime();

    printf("Sum = %f\n", sum);
    printf("Execution Time (reduction) = %f seconds\n", end - start);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was
```

```
nithish@LAPTOP-A2FOJ865:~$ nano reduction_good.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp reduction_good.c -o reduction_good
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=1
./reduction_good
Sum = 2499999975000000.000000
Execution Time (reduction) = 0.109086 seconds
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./reduction_good
Sum = 2499999975000000.000000
Execution Time (reduction) = 0.033849 seconds
nithish@LAPTOP-A2FOJ865:~$
```

Task 2: Load Imbalance and Scheduling

Create a file named imbalance.c.
Implement a loop where:
One iteration performs heavy computation
Other iterations perform small computation
Run with default scheduling.
Record execution time.

Now modify to:
    #pragma omp parallel for schedule(dynamic)
Run again and record time.

```
  GNU nano 7.2                                                    imbalance.c *
#include <stdio.h>
#include <omp.h>

int main() {

    int i, j;
    long long work;
    int N = 20;

    double start = omp_get_wtime();

    #pragma omp parallel for
    for(i = 0; i < N; i++) {

        work = 0;

        // One heavy iteration (load imbalance)
        if(i == 0) {
            for(j = 0; j < 500000000; j++)
                work += j % 5;
        }
        // light iterations
        else {
            for(j = 0; j < 5000000; j++)
                work += j % 5;
        }

        printf("Thread %d finished iteration %d\n",
               omp_get_thread_num(), i);
    }

    double end = omp_get_wtime();

    printf("Execution Time (default scheduling) = %f seconds\n",
           end - start);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-Q Previous
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6 Copy       ^Q Where Was    M-W Next
```

```
nithish@LAPTOP-A2FOJ865:~$ nano imbalance.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp imbalance.c -o imbalance
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./imbalance
Thread 3 finished iteration 15
Thread 1 finished iteration 5
Thread 2 finished iteration 10
Thread 3 finished iteration 16
Thread 3 finished iteration 17
Thread 2 finished iteration 11
Thread 1 finished iteration 6
Thread 3 finished iteration 18
Thread 2 finished iteration 12
Thread 1 finished iteration 7
Thread 3 finished iteration 19
Thread 2 finished iteration 13
Thread 1 finished iteration 8
Thread 2 finished iteration 14
Thread 1 finished iteration 9
Thread 0 finished iteration 0
Thread 0 finished iteration 1
Thread 0 finished iteration 2
Thread 0 finished iteration 3
Thread 0 finished iteration 4
Execution Time (default scheduling) = 0.432136 seconds
nithish@LAPTOP-A2FOJ865:~$
```

```
nithish@LAPTOP-A2FOJ865:~$ nano imbalance.c
nithish@LAPTOP-A2FOJ865:~$ gcc -O2 -fopenmp imbalance.c -o imbalance
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./imbalance
Thread 3 finished iteration 3
Thread 1 finished iteration 1
Thread 2 finished iteration 2
Thread 1 finished iteration 5
Thread 3 finished iteration 4
Thread 2 finished iteration 6
Thread 2 finished iteration 9
Thread 1 finished iteration 7
Thread 3 finished iteration 8
Thread 2 finished iteration 10
Thread 3 finished iteration 12
Thread 1 finished iteration 11
Thread 2 finished iteration 13
Thread 3 finished iteration 14
Thread 1 finished iteration 15
Thread 2 finished iteration 16
Thread 1 finished iteration 18
Thread 3 finished iteration 17
Thread 2 finished iteration 19
Thread 0 finished iteration 0
Execution Time (dynamic scheduling) = 0.930937 seconds
nithish@LAPTOP-A2FOJ865:~$
```

Task 3: Performance Analysis Questions

Answer the following:

1. Why does using critical reduce scalability?

When we use a critical section, only one thread is allowed to access that part of the code at a time. So even though multiple threads are created, they end up waiting in a queue to update the shared variable. As the number of threads increases, the waiting time also increases, which limits the benefit of parallel execution.

2. How does reduction improve performance?

With reduction, each thread works on its own temporary copy of the variable instead of sharing one common variable. This means threads can run independently without waiting for each other. At the end, OpenMP combines all results automatically, which makes the program faster and more efficient.

3. What is load imbalance?

Load imbalance happens when some threads get more work while others get less. The threads that finish early remain idle while one or two threads are still busy. Because not all CPU resources are used equally, the overall performance decreases.

4. Why does dynamic scheduling improve performance in uneven workloads?

Dynamic scheduling distributes work while the program is running instead of assigning everything at the beginning. If a thread finishes early, it immediately receives new work. This

keeps all threads busy and reduces idle time, which improves overall performance when workloads are uneven.

5. Identify the hotspot section in your program and justify.
The hotspot in the program is the heavy computation loop where one iteration performs much more work than the others. This part takes the longest time to execute and dominates the total runtime, making it the main performance bottleneck.